KHATA VOICE (ఖాత వయిస్ / खाता वॉयस)

A Multilingual Real-Time Voice Ledger for Indian Kirana & Retail Shopkeepers

Challenge:	The Great Indian AI Internship Challenge (Gnani.ai)
Candidate / Author:	Chari (Warangal, Telangana — Tier 2/3 Town Category)
Core Technologies:	Gnani Prisma v3.0 (STT), Soundbox TTS Engine, WhatsApp Statements, Kirana Analytics
Supported Languages:	Telugu (te-IN), Hindi (hi-IN), Indian English (en-IN), Code-mixed
Security & Multi-Tenancy:	Per-Shopkeeper Isolated Ledgers, PIN Authentication, Persistent Language Profiles
Target Awards:	Regional Languages, Hinglish/Code-mixed, Noisy Audio, Tier 2/3, Best 60-Sec Demo
Project Status:	Phase 0 to 5 Complete — Production-Grade Multilingual Voice Fintech Platform

1. Executive Summary & Problem Statement

Across India's vast landscape of over 12 million small kirana stores, traditional retail commerce runs on credit (udhaar / baaki / బాకీ). Shopkeepers manage these daily transactions using physical paper notebooks (bahi khata / బహీ ఖాత). While informal credit builds customer loyalty, manual bookkeeping is fraught with operational challenges: it is slow during rush hours, paper books get torn, lost, or water-damaged, calculating balances requires tedious manual arithmetic, and entering transactions into existing mobile apps demands tedious manual typing that disrupts customer service.

Khata Voice transforms this experience into a zero-friction, voice-first interaction with multi-tenant shopkeeper accounts, automatic Soundbox voice announcements, 1-click WhatsApp payment reminders, and visual business analytics. Each shopkeeper creates an account with their preferred language (Telugu, Hindi, or English). When logged in, the entire interface (metrics, labels, buttons, and ledger) adapts natively to that language. The shopkeeper speaks naturally (e.g., 'రాము 500 రూపయలు బియ్యం', 'రాము ఎంత బాకీ', or 'రాము 200 చిల్లించడు'). The system transcribes audio via Gnani.ai Prisma v3.0 STT, extracts customer names, provisions (సరుకులు), and amounts, presents a human-in-the-loop review dialogue, speaks aloud a Paytm-style audio confirmation, and commits the entry to that shopkeeper's private, isolated ledger with instant balance calculation.

2. Gnani.ai Challenge Structure & Official Guidelines

- Internship Offer: Up to 100 paid internships starting January 1, 2027, with stipends up to ₹1,00,000 per month.
- 10 Merit Awards (₹50,000 each): Independent awards targeting Regional Languages, Code-mixed voice, Noisy audio, Real-world impact, Solo/Duo project, Tier 2/3 town entries, and Best 60-second demo.
- Hard Submission Cutoff: November 10, 2026 at 11:59 PM IST.
- Deliverables: 1) Public 60s demo video on LinkedIn/X with tags #GnaniAI and #GreatIndianAIInternshipChallenge; 2) Workspace documentation; 3) Submission Form.

3. Advanced Architecture & Enterprise Feature Set

Khata Voice integrates six enterprise-grade pillars engineered specifically for Indian retail:

Layer / Feature	Module	Implementation & Technology	Key Functionality
Authentication	auth.py	users.json with PIN security	Multi-shopkeeper signup/login, persistent language selection (te/hi/en)
Data Isolation	ledger.py	ledger_{username}.json per shop	Dynamic get_ledger_file(user); zero cross-tenant data leakage.
STT Engine	stt.py	Gnani Prisma v3.0 REST Client	Sub-second ASR with dynamic keyterm boosting from shopkeeper's c
NLP Extractor	extract.py	Provisions & Fuzzy Matcher	Normalizes provisions ([redacted]/BM, [redacted], [redacted]) and match
Soundbox TTS	tts.py	Web Speech Audio Component	Auto-announces confirmed transactions like Paytm/PhonePe soundbo
WhatsApp Reminders	tts.py	URL Scheme Generator	1-click pre-filled WhatsApp debt reminder with provisions and balance
Kirana Analytics	app.py	Streamlit Native Bar Charts	Visualizes top debtors, recovery rate %, and most demanded provisio

4. Complete Production Source Code

Below is the complete, unabridged production source code for every file in the Khata Voice repository, including authentication, soundbox TTS, and analytics logic.

File: auth.py

Role: Multi-shopkeeper authentication engine with PIN verification, account creation, and language persistence.

```
import json
import os
from datetime import date

USERS_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "users.json")

DEFAULT_USERS = {
    "chari": {
        "shop_name": "Chari Kirana Store (శ్రీ లక్ష్మి కిరణ)",
        "owner_name": "Chari",
        "pin": "1234",
        "language": "te",
        "created_at": str(date.today())
    }
}


def load_users():
    if not os.path.exists(USERS_FILE):
        save_users(DEFAULT_USERS)
        return dict(DEFAULT_USERS)
    try:
        with open(USERS_FILE, "r", encoding="utf-8") as f:
            return json.load(f)
    except Exception:
        return dict(DEFAULT_USERS)


def save_users(users):
    with open(USERS_FILE, "w", encoding="utf-8") as f:
```

```
        json.dump(users, f, ensure_ascii=False, indent=2)


def register_user(username, shop_name, owner_name, pin, language="te"):
    users = load_users()
    key = username.strip().lower()
    if not key:
        return False, "Username cannot be empty."
    if key in users:
        return False, f"Account '{username}' already exists. Please login."
    if len(str(pin).strip()) < 4:
        return False, "PIN must be at least 4 digits."

    users[key] = {
        "shop_name": shop_name.strip() or f"{username}'s Shop",
        "owner_name": owner_name.strip() or username,
        "pin": str(pin).strip(),
        "language": language if language in ("te", "hi", "en") else "te",
        "created_at": str(date.today())
    }
    save_users(users)
    return True, f"Account created successfully for '{users[key]['shop_name']}'!"


def authenticate_user(username, pin):
    users = load_users()
    key = username.strip().lower()
    if key not in users:
        return False, None, "Account not found. Please sign up first."
    user = users[key]
    if user["pin"] != str(pin).strip():
        return False, None, "Incorrect PIN. Please try again."
    user_copy = dict(user)
    user_copy["username"] = key
    return True, user_copy, "Login successful!"


def update_language(username, language):
    users = load_users()
    key = username.strip().lower()
    if key in users and language in ("te", "hi", "en"):
        users[key]["language"] = language
        save_users(users)
        return True
    return False


if __name__ == "__main__":
    print("Testing auth...")
    ok, u, msg = authenticate_user("chari", "1234")
    print("Default login:", ok, u["shop_name"], u["language"])
    reg_ok, reg_msg = register_user("ravi_kirana", "Ravi General Stores", "Ravi", "5678", "hi")
    print("Register:", reg_ok, reg_msg)
    auth_ok, auth_u, auth_msg = authenticate_user("ravi_kirana", "5678")
    print("New login:", auth_ok, auth_u["language"])
```

File: tts.py

Role: Smart Soundbox audio synthesis, multilingual transaction announcement formatting, and WhatsApp statement links.

```
import urllib.parse


def format_announcement(intent, customer, amount=None, remaining=None, provision=None, lang="te"):
    """
    Generates natural, soundbox-style announcements for retail store transactions.
    """
    amt_str = str(abs(amount)) if amount is not None else ""
    rem_str = str(remaining) if remaining is not None else ""
    cust_str = customer or "కస్టమర్"

    if lang == "te":
        if intent == "add":
            prov_txt = f" ({provision})" if provision and provision != "—" else ""
            return f"{cust_str} ఖాతలో {amt_str} రూపయలు నమోదయ్యయి{prov_txt}."
        elif intent == "pay":
            tail = "ఖాత పూర్తిగ క్లియర్ అయ్యింది." if remaining == 0 else f"మిగిలిన బకీ {rem_str} రూపయలు."
            return f"{cust_str} {amt_str} రూపయలు చెల్లించరు. {tail}"
        elif intent == "ask":
            return f"{cust_str} బకీ మొత్తం {rem_str} రూపయలు."
        elif intent == "correct":
            return f"చివరి ఎంట్రీ మొత్తం {amt_str} రూపయలుగ సవరించబడింది."
        return "లావదేవీ పూర్తయ్యింది."

    elif lang == "hi":
        if intent == "add":
            prov_txt = f" ({provision})" if provision and provision != "—" else ""
            return f"{cust_str} के खाते में {amt_str} रुपये दर्ज हुए{prov_txt}."
        elif intent == "pay":
            tail = "खाता पूरी तरह चुकता हो गया." if remaining == 0 else f"बाकी बचे {rem_str} रुपये."
            return f"{cust_str} ने {amt_str} रुपये चुकाए. {tail}"
        elif intent == "ask":
            return f"{cust_str} का कुल बाकी {rem_str} रुपये है."
        elif intent == "correct":
            return f"अंतमि एंट्री {amt_str} रुपये पर संशोधति की गई."
        return "लेन-देन दर्ज हुआ."

    else:  # en
        if intent == "add":
            prov_txt = f" for {provision}" if provision and provision != "—" else ""
            return f"{amt_str} rupees added to {cust_str}'s account{prov_txt}."
        elif intent == "pay":
            tail = "Account is fully settled." if remaining == 0 else f"Balance now {rem_str} rupees."
            return f"{cust_str} paid {amt_str} rupees. {tail}"
        elif intent == "ask":
            return f"{cust_str} owes {rem_str} rupees."
        elif intent == "correct":
            return f"Last entry corrected to {amt_str} rupees."
        return "Transaction recorded successfully."


def soundbox_html(text, lang="te"):
    """
    Renders an interactive, auto-playing Soundbox announcement widget with Web Speech API.
    """
    lang_code = "te-IN" if lang == "te" else ("hi-IN" if lang == "hi" else "en-IN")
    escaped_text = text.replace('"', '\\"').replace("'", "\\'")

    return f"""
    <div style="background: linear-gradient(135deg, #1A365D, #2B6CB0); color: white; padding: 14px 20px; border-radius: 12px; margin: 12px 0 16px 0; box-shad
ow: 0 4px 12px rgba(0,0,0,0.15); display: flex; align-items: center; justify-content: space-between; font-family: sans-serif;">
```

```
        <div style="display: flex; align-items: center; gap: 12px;">
            <div style="background: rgba(255,255,255,0.2); width: 40px; height: 40px; border-radius: 50%; display: flex; align-items: center; justify-content: center; fo
nt-size: 22px;">

            </div>
            <div>
                <div style="font-size: 11px; text-transform: uppercase; letter-spacing: 1px; color: #CBD5E0; font-weight: 600;">
                    Khata Voice Smart Soundbox
                </div>
                <div style="font-size: 15px; font-weight: 500; margin-top: 2px;">
                    "{text}"
                </div>
            </div>
        </div>
        <button onclick="speakSoundbox()" style="background: #ECC94B; color: #1A202C; border: none; padding: 8px 16px; border-radius: 8px; font-size: 13px; fo
nt-weight: bold; cursor: pointer; transition: 0.2s; display: flex; align-items: center; gap: 6px;">
            <span></span> Replay Audio
        </button>
    </div>
    <script>
    function speakSoundbox() {{
        if ('speechSynthesis' in window) {{
            window.speechSynthesis.cancel();
            var utterance = new SpeechSynthesisUtterance("{escaped_text}");
            utterance.lang = "{lang_code}";
            utterance.rate = 0.95;
            window.speechSynthesis.speak(utterance);
        }}
    }}
    // Auto-announce on render
    setTimeout(speakSoundbox, 400);
    </script>
    """


def generate_whatsapp_link(customer_name, balance, provisions, shop_name="శ్రీ లక్ష్మి కిరణా", lang="te"):
    """
    Generates a pre-filled WhatsApp payment reminder link with exact balance and provisions.
    """
    if lang == "te":
        msg = f"నమస్కారం {customer_name} గారు, మీ {shop_name} దుకాణం బాకీ మొత్తం ₹{balance:,} (సరుకులు: {provisions}). దయచేసి వీలైనంత త్వరగా చెల్లిం
చగలరు. ధన్యవాదములు!"
    elif lang == "hi":
        msg = f"नमस्ते {customer_name} जी, आपकी {shop_name} की कुल बाकी राशि ₹{balance:,} है (सामान: {provisions}). कृपया जल्द से जल्द भुगतान करें. धन्यवाद!"
    else:
        msg = f"Hello {customer_name}, your outstanding balance at {shop_name} is ₹{balance:,} (Provisions: {provisions}). Kindly clear the dues at your earliest con
venience. Thank you!"

    encoded_msg = urllib.parse.quote(msg)
    return f"https://wa.me/?text={encoded_msg}"


if __name__ == "__main__":
    t_msg = format_announcement("add", "రము", 500, provision="బియ్యం (Rice)", lang="te")
    print("Telugu Add:", t_msg)
    p_msg = format_announcement("pay", "రము", 200, remaining=300, lang="te")
    print("Telugu Pay:", p_msg)
    wa_url = generate_whatsapp_link("రము", 400, "బియ్యం, పంచదర")
    print("WhatsApp Link:", wa_url)
```

File: stt.py

Role: Gnani.ai Prisma v3.0 REST API client with keyterms boosting and multi-language support (te-IN, hi-IN, en-IN).

```
import requests, os
from dotenv import load_dotenv

load_dotenv()

def get_api_key():
    key = os.environ.get("GNANI_API_KEY")
    if not key:
        raise RuntimeError("GNANI_API_KEY is not set. Please set it in your environment or in a .env file.")
    return key

def transcribe_audio(audio_path, languages="te-IN,hi-IN,en-IN", keyterms=None):
    key = get_api_key()
    headers = {"X-API-Key-ID": key}
    data = {"model": "prisma-v3.0", "language_code": languages, "format": "transcribe"}
    if keyterms:
        data["keyterms"] = ",".join(keyterms)
    with open(audio_path, "rb") as f:
        response = requests.post("https://api.vachana.ai/stt/v3", headers=headers, files={"audio_file": f}, data=data, timeout=60)
    if response.status_code != 200:
        raise RuntimeError(f'STT failed: {response.status_code} {response.text}')
    return response.json()

if __name__ == "__main__":
    result = transcribe_audio("telugu1.m4a")
    print(result["transcript"])
    print(result.get("detected_language_code"))
```

File: intent.py

Role: Deterministic multilingual intent classifier (add, ask, correct, pay) with word-boundary and stem matching.

```
import re

ASK_WORDS = ["how much", "balance", "owes", "total", "कितना", "बाकी", "हिसाब", "ఎంత", "బాకీ", "మొత్తం"]
CORRECT_WORDS = ["make it", "change", "wrong", "correct", "mistake", "गलत", "बदल", "सुधार", "తప్పు", "మార్చు", "సరిచేయి"]
PAY_WORDS = ["paid", "pay", "payment", "cleared", "settled", "returned", "gave back",
    "चुका", "भुगतान", "वापस", "जमा",
    "చెల్లించ", "చెల్లింపు", "కట్టి", "తీర్చ"]
FULL_WORDS = ["full", "fully", "all", "everything", "whole", "balance",
    "पूरा", "पूरी", "सारा", "మొత్తం", "అంత", "పూర్తి"]


def contains(text, words):
    lowered = text.lower()
    padded = " " + re.sub(r"[^\w\s]", " ", lowered) + " "
    for w in words:
        if w.isascii():
            if " " + w + " " in padded:
                return True
        elif w in lowered:
            return True
    return False


def detect_intent(text):
    if contains(text, CORRECT_WORDS):
        return "correct"
    if contains(text, PAY_WORDS):
        return "pay"
    if contains(text, ASK_WORDS):
        return "ask"
```

```
    return "add"


if __name__ == "__main__":
    samples = [
        "Ramu 500 credit rice",
        "how much does Ramu owe",
        "make it 600",
        "रामू का कितना बाकी है",
        "రము ఎంత బాకీ",
        "Ramu paid 200",
        "Ramu paid full",
        "रामू ने 200 चुकाए",
        "రము 200 చెల్లించడు",
    ]
    for s in samples:
        print(s, "->", detect_intent(s))
```

File: extract.py

Role: Entity extraction engine: finds customers, numerical amounts, and provision items (BM, Panchadar, Nune, etc.).

```
import re
import difflib

FILLER = ["credit", "rupees", "rupee", "rs", "took", "taken", "for", "to", "of", "the", "and",
    "paid", "pay", "payment", "gave", "returned", "cleared", "settled", "back", "full", "fully", "all", "everything",
    "రూపాయలు", "రూపాయల", "రూపాయ", "కి", "కు",
    "చెల్లించడు", "చెల్లించారు", "చెల్లింపు", "కట్టడు", "కట్టారు",
    "रुपये", "रुपए", "रुपया", "का", "की", "के", "को", "ने",
    "चुकाया", "चुकाए", "भुगतान", "वापस", "जमा"]
PUNCT = ".,!?:;₹।"

ITEM_ALIASES = {
    "bm": "బియ్యం (Rice)",
    "biyyam": "బియ్యం",
    "rice": "బియ్యం (Rice)",
    "panchadar": "పంచదార",
    "panchadara": "పంచదార",
    "sugar": "పంచదార (Sugar)",
    "pappu": "కందిపప్పు",
    "kandi pappu": "కందిపప్పు",
    "dal": "పప్పు (Dal)",
    "nune": "నూనె",
    "oil": "నూనె (Oil)",
    "sabbu": "సబ్బు",
    "soap": "సబ్బు (Soap)",
    "aata": "గోధుమ పిండి (Aata)",
    "pindi": "పిండి",
    "tea": "టీ పొడి (Tea)",
    "tea podi": "టీ పొడి",
}
```

```
def words_of(chunk):
    return [w.strip(PUNCT) for w in chunk.split() if w.strip(PUNCT)]


def find_amount(text):
    match = re.search(r"\d[\d,]*", text)
    if match:
        return int(match.group().replace(",", ""))
    return None


def find_customer(text, known):
    tokens = [w.lower() for w in words_of(text)]
    for name in sorted(known, key=lambda n: len(words_of(n)), reverse=True):
        parts = [p.lower() for p in words_of(name)]
        for i in range(len(tokens) - len(parts) + 1):
            if tokens[i:i + len(parts)] == parts:
                return name, None
    match = re.search(r"\d[\d,]*", text)
    before = text[:match.start()] if match else ""
    words = [w for w in words_of(before) if w.lower() not in FILLER]
    if not words:
        return None, None
    name = " ".join(words)
    name = name.title() if name.isascii() else name
    known_lower = {k.lower(): k for k in known}
    close = difflib.get_close_matches(name.lower(), list(known_lower), n=1, cutoff=0.75)
    return name, (known_lower[close[0]] if close else None)


def find_item(text, customer):
    match = re.search(r"\d[\d,]*", text)
    if not match:
        return None
    skip = set(w.lower() for w in (customer or "").split())
    items = [w for w in words_of(text[match.end():]) if w.lower() not in FILLER and w.lower() not in skip]
    if not items:
        return None
    raw_item = " ".join(items)
    lowered = raw_item.lower()
    if lowered in ITEM_ALIASES:
        return ITEM_ALIASES[lowered]
    return raw_item


def extract_entry(text, known=None):
    known = known or []
    customer, similar = find_customer(text, known)
    return {
        "customer": customer,
        "amount": find_amount(text),
        "item": find_item(text, customer),
        "similar_to": similar,
    }


if __name__ == "__main__":
    known = ["Ramu", "సీత"]
```

```
    samples = [
        "Ramu 500 credit rice",
        "Sravya 300 rupees sugar",
        "నీమా 300 రూపయలు బియ్యం",
        "రము 500 bm",
        "రము 200 panchadar",
        "రము 150 nune",
        "राम् 300 रुपये आटा",
        "how much does Ramu owe",
    ]
    for s in samples:
        print(s, "->", extract_entry(s, known))
```

File: ledger.py

Role: Multi-tenant ledger engine with per-user file isolation (ledger_{user}.json), financial analytics, and provisions.

```
import json
import os
from datetime import date

BASE_DIR = os.path.dirname(os.path.abspath(__file__))


def get_ledger_file(username=None):
    if not username:
        return os.path.join(BASE_DIR, "ledger.json")
    safe_name = "".join(c for c in username.strip().lower() if c.isalnum() or c in "_-")
    return os.path.join(BASE_DIR, f"ledger_{safe_name}.json")


def load_entries(username=None):
    filepath = get_ledger_file(username)
    if not os.path.exists(filepath):
        # Pre-populate demo records ONLY for chari demo account
        if username and username.strip().lower() == "chari":
            demo = [
                {"customer": "నీత", "amount": 300, "item": "పంచదర", "type": "credit", "date": str(date.today())},
                {"customer": "సమంత", "amount": 300, "item": "బియ్యం", "type": "credit", "date": str(date.today())}
            ]
            save_entries(demo, username)
            return demo
        # Any other new account gets an empty ledger
        save_entries([], username)
        return []
    try:
        with open(filepath, "r", encoding="utf-8") as f:
```

```
            return json.load(f)
    except Exception:
        return []


def save_entries(entries, username=None):
    filepath = get_ledger_file(username)
    with open(filepath, "w", encoding="utf-8") as f:
        json.dump(entries, f, ensure_ascii=False, indent=2)


def add_entry(entry, username=None):
    entries = load_entries(username)
    new_entry = dict(entry)
    new_entry.setdefault("type", "credit")
    new_entry["date"] = str(date.today())
    entries.append(new_entry)
    save_entries(entries, username)
    return new_entry


def add_payment(customer, amount, username=None):
    return add_entry({"customer": customer, "amount": -abs(amount), "item": "payment", "type": "payment"}, username)


def balance(customer, username=None):
    return sum(e["amount"] for e in load_entries(username) if e.get("customer") == customer and e.get("amount") is not None)


def all_balances(username=None):
    totals = {}
    for e in load_entries(username):
        if e.get("customer") and e.get("amount") is not None:
            totals[e["customer"]] = totals.get(e["customer"], 0) + e["amount"]
    return totals


def customer_provisions(customer, username=None):
    entries = load_entries(username)
    items = []
    for e in entries:
        if e.get("customer") == customer and e.get("item") and e.get("item") != "payment":
            if e["item"] not in items:
                items.append(e["item"])
    return ", ".join(items) if items else "—"


def customers_summary(username=None):
    totals = all_balances(username)
    entries = load_entries(username)
    items_map = {}
    for e in entries:
        c = e.get("customer")
        it = e.get("item")
        if c and it and it != "payment":
            if c not in items_map:
                items_map[c] = []
            if it not in items_map[c]:
                items_map[c].append(it)
    summary = []
```

```
    for name, total in totals.items():
        provisions = ", ".join(items_map.get(name, [])) or "—"
        status = "Settled (క్లియర్)" if total == 0 else ("Owes (బాకీ)" if total > 0 else "Advance (అడ్వాన్స్)")
        summary.append({
            "Customer": name,
            "Balance": total,
            "Provisions": provisions,
            "Status": status
        })
    return summary


def customers(username=None):
    return sorted({e["customer"] for e in load_entries(username) if e.get("customer")})


def correct_last(new_amount, username=None):
    entries = load_entries(username)
    if not entries:
        return None
    last = entries[-1]
    sign = -1 if last.get("amount") and last["amount"] < 0 else 1
    last["amount"] = sign * abs(new_amount)
    save_entries(entries, username)
    return last


# --- ADVANCED KIRANA ANALYTICS ---
def financial_metrics(username=None):
    entries = load_entries(username)
    total_credit = sum(e["amount"] for e in entries if e.get("amount") and e["amount"] > 0)
    total_collected = sum(abs(e["amount"]) for e in entries if e.get("amount") and e["amount"] < 0)
    current_outstanding = total_credit - total_collected
    recovery_rate = (total_collected / total_credit * 100) if total_credit > 0 else 100.0
    return {
        "total_credit": total_credit,
        "total_collected": total_collected,
        "current_outstanding": current_outstanding,
        "recovery_rate": round(recovery_rate, 1)
    }


def provisions_stats(username=None):
    entries = load_entries(username)
    counts = {}
    for e in entries:
        it = e.get("item")
        if it and it != "payment":
            counts[it] = counts.get(it, 0) + 1
    return counts


if __name__ == "__main__":
    print("Chari metrics:", financial_metrics("chari"))
    print("Chari provisions:", provisions_stats("chari"))
```

File: app.py

Role: Streamlit web application with soundbox audio readback, WhatsApp reminders, business analytics, and voice ledger UI.

```
import os
import importlib
import streamlit as st
import streamlit.components.v1 as components
import pandas as pd
from dotenv import load_dotenv

import pipeline
import ledger
import auth
import tts

# Force-reload modules on every rerun so no stale in-memory module state is ever used
importlib.reload(ledger)
importlib.reload(pipeline)
importlib.reload(auth)
importlib.reload(tts)

load_dotenv()

st.set_page_config(page_title="Khata Voice — Voice Ledger", page_icon="", layout="wide")

# --- I18N DICTIONARY FOR LOCALIZATION ---
I18N = {
    "te": {
        "app_title": " ఖత వయిస్ (Khata Voice)",
        "app_subtitle": "చీరు వ్యాపారుల రియల్-టైమ్ వయిస్ లెడ్జర్ (కిరణ & జనరల్ స్టోర్స్)",
        "outstanding": "మొత్తం బకీ (Outstanding)",
        "active_cust": "ఖాతదరులు (బకీ ఉన్నవరు)",
        "settled_cust": "క్లియర్ అయిన ఖతలు (Settled)",
        "step1_title": "1. వయిస్ రికర్డ్ చేయండి లేద ఆడియో అప్‌లోడ్ చేయండి",
        "record_hint": "ఎంట్రీ మట్లాడండి (30 సెకన్ల లోపు)",
        "or_upload": "లేద ఆడియో ఫైల్ అప్‌లోడ్ చేయండి",
        "btn_transcribe": " వయిస్ అర్థం చేసుకోండి",
        "step2_title": "2. ఎంట్రీని తనిఖీ చేసి ధృవీకరించండి",
        "heard": "వినబడిన మాటలు:",
        "intent": "గుర్తించిన ఉద్దేశం:",
        "lang": "భష:",
        "customer": "ఖాతదరు:",
        "amount": "మొత్తం:",
        "provision": "సరుకు:",
        "btn_confirm": " లెడ్జర్‌లో నమోదు చేయండి (Confirm)",
        "btn_cancel": " రద్దు చేయండి (Cancel)",
        "step3_title": "3. ఈ దుకణం ఖత పుస్తకం (Shop Ledger)",
        "tab_balances": "ఖాతదరులు & సరుకులు (Balances)",
        "tab_history": "పూర్తి లవదేవీల చరిత్ర (History)",
        "tab_analytics": " దుకణ విశ్లేషణ (Analytics)",
        "col_customer": "ఖాతదరు (Customer)",
        "col_balance": "బకీ ₹ (Balance)",
        "col_provisions": "తీసుకున్న సరుకులు (Provisions)",
        "col_status": "స్థితి (Status)",
        "btn_export": " లెడ్జర్ CSV డౌన్‌లోడ్ చేయండి",
        "status_owes": "బకీ (Owes)",
        "status_settled": "క్లియర్ (Settled)",
        "status_advance": "అడ్వన్స్ (Advance)",
        "quick_demo": " త్వరిత డెమో నమూనలు",
        "shop_customers_title": " ఈ దుకణం ఖాతదరులు (Your Customers)",
        "no_customers": "ఇంక ఖాతదరులు లేరు (No customers yet)",
        "logout": "లాగౌట్ (Log Out)",
        "switch_lang": "భషను మర్చండి (Language)",
```

```
    "empty_ledger": "ఈ దుకాణంలో ఇంకా ఎలాంటి ఎంట్రీలు నమోదు కలేదు. పైన వయిస్ మట్లాడి రికర్డ్ చేయండి!",
    "search_label": " ఖాతదరు లేద సరుకు వేరు వెతకండి (Search)",
    "send_whatsapp": " WhatsApp రిమైండర్ పంపండి",
    "total_credit": "మొత్తం ఇచ్చిన అప్పు (Total Credit)",
    "total_collected": "వసూలు అయిన నగదు (Collected)",
    "recovery_rate": "రికవరీ రేటు (Recovery Rate)",
    "top_debtors_title": "అత్యధిక బకీ ఉన్న ఖాతదరులు",
    "top_provisions_title": "అత్యధికంగ అమ్ముడైన సరుకులు",
},
"hi": {
    "app_title": " खाता वॉयस (Khata Voice)",
    "app_subtitle": "दुकानदारों के लिए रयिल-टाइम वॉयस बही-खाता (करिाना और जनरल स्टोर)",
    "outstanding": "कुल बाकी उधार (Outstanding)",
    "active_cust": "सक्रयि ग्राहक (उधार वाले)",
    "settled_cust": "हसिाब चुकता (Settled)",
    "step1_title": "1. बोलकर या ऑडयिो अपलोड करके रकिॉर्ड करें",
    "record_hint": "एंट्री बोलें (30 सेकंड के अंदर)",
    "or_upload": "या ऑडयिो फ़ाइल अपलोड करें",
    "btn_transcribe": " आवाज़ को समझें (Transcribe)",
    "step2_title": "2. एंट्री की जांच करें और पुष्टि करें",
    "heard": "सुना गया:",
    "intent": "पहचाना गया उद्देश्य:",
    "lang": "भाषा:",
    "customer": "ग्राहक:",
    "amount": "रुपये:",
    "provision": "सामान / राशन:",
    "btn_confirm": " खाते में दर्ज करें (Confirm)",
    "btn_cancel": " रद्द करें (Cancel)",
    "step3_title": "3. इस दुकान का बही-खाता (Shop Ledger)",
    "tab_balances": "ग्राहकों का हसिाब और राशन (Balances)",
    "tab_history": "लेन-देन का पूरा इतहिास (History)",
    "tab_analytics": " दुकान का वशि्लेषण (Analytics)",
    "col_customer": "ग्राहक (Customer)",
    "col_balance": "बाकी रुपये ₹ (Balance)",
    "col_provisions": "सामान / राशन (Provisions)",
    "col_status": "स्थति (Status)",
    "btn_export": " बही-खाता CSV डाउनलोड करें",
    "status_owes": "बाकी (Owes)",
    "status_settled": "चुकता (Settled)",
    "status_advance": "जमा (Advance)",
    "quick_demo": " तुरंत डेमो के उदाहरण",
    "shop_customers_title": " इस दुकान के ग्राहक (Your Customers)",
    "no_customers": "अभी कोई ग्राहक नहीं है (No customers yet)",
    "logout": "लॉग आउट (Log Out)",
    "switch_lang": "भाषा बदलें (Language)",
    "empty_ledger": "इस दुकान में अभी कोई एंट्री दर्ज नहीं है। ऊपर बोलकर रकिॉर्ड करें!",
    "search_label": " ग्राहक या सामान का नाम खोजें (Search)",
    "send_whatsapp": " WhatsApp तकादा / रमिाइंडर भेजें",
    "total_credit": "कुल दयिा गया उधार (Total Credit)",
    "total_collected": "कुल वसूल हुआ पैसा (Collected)",
    "recovery_rate": "वसूली दर (Recovery Rate)",
    "top_debtors_title": "सबसे ज्यादा बाकी वाले ग्राहक",
    "top_provisions_title": "सबसे ज्यादा बकिने वाला सामान",
},
"en": {
    "app_title": " Khata Voice",
    "app_subtitle": "Multilingual Real-Time Voice Ledger for Kirana & Retail Stores",
    "outstanding": "Total Outstanding",
    "active_cust": "Active Customers with Balance",
    "settled_cust": "Settled Accounts",
```

```
        "step1_title": "1. Speak or Upload Audio",
        "record_hint": "Record an entry (under 30 seconds)",
        "or_upload": "Or upload an audio clip",
        "btn_transcribe": " Transcribe & Understand Voice",
        "step2_title": "2. Review & Confirm Entry",
        "heard": "Heard:",
        "intent": "Detected Intent:",
        "lang": "Language:",
        "customer": "Customer:",
        "amount": "Amount:",
        "provision": "Provision:",
        "btn_confirm": " Confirm & Save to Khata",
        "btn_cancel": " Cancel",
        "step3_title": "3. This Shop's Ledger",
        "tab_balances": "Customer Balances & Provisions",
        "tab_history": "Transaction History",
        "tab_analytics": " Kirana Analytics",
        "col_customer": "Customer",
        "col_balance": "Balance (₹)",
        "col_provisions": "Provisions Taken",
        "col_status": "Status",
        "btn_export": " Export Ledger to CSV",
        "status_owes": "Owes",
        "status_settled": "Settled",
        "status_advance": "Advance",
        "quick_demo": " Quick Demo / Test Phrases",
        "shop_customers_title": " This Shop's Customers",
        "no_customers": "No customers registered yet",
        "logout": "Log Out",
        "switch_lang": "Language",
        "empty_ledger": "No entries recorded yet for this shop. Record a voice note above or click a demo sample on the sidebar!",
        "search_label": " Search customer or provision name",
        "send_whatsapp": " Send WhatsApp Reminder",
        "total_credit": "Total Credit Given",
        "total_collected": "Total Debt Recovered",
        "recovery_rate": "Debt Recovery Rate",
        "top_debtors_title": "Top Customers by Outstanding Debt",
        "top_provisions_title": "Most Demanded Provisions",
    }
}

# --- AUTHENTICATION GATEWAY ---
if "user" not in st.session_state:
    st.session_state["user"] = None

user = st.session_state["user"]

# If user is not logged in, show Sign In / Sign Up portal
if user is None:
    st.title(" Khata Voice (ఖాత వయిస్ / खाता वॉयस)")
    st.caption("Powered by **Gnani.ai Prisma v3.0** | The Great Indian AI Internship Challenge")
    st.markdown("---")

    col_auth_left, col_auth_right = st.columns([1.2, 1])

    with col_auth_left:
        st.subheader(" Welcome to Khata Voice")
        st.write(
            "Every shopkeeper has their own private, independent khata ledger. "
            "Customer details and credit entries belong exclusively to that shopkeeper and are never visible to any other shop."
```

```
        )
        st.info(" **Instant Demo Access:** A pre-configured shopkeeper account is ready. Just click below!")
        if st.button(" Quick Demo Login as Chari (శ్రీ లక్ష్మి కిరణ — తెలుగు)", type="primary"):
            ok, demo_u, msg = auth.authenticate_user("chari", "1234")
            if ok:
                st.session_state.clear()
                st.session_state["user"] = demo_u
                st.rerun()

    with col_auth_right:
        auth_tab_login, auth_tab_signup = st.tabs([" లాగిన్ (Login)", "కొత్త ఖాత (Sign Up)"])

        with auth_tab_login:
            st.markdown("#### దుకాణదారు లాగిన్ (Shopkeeper Login)")
            login_user = st.text_input("ఖాత పేరు / Username", value="chari", key="l_user")
            login_pin = st.text_input("4-అంకెల పిన్ / PIN", value="1234", type="password", key="l_pin")

            if st.button("ప్రవేశించండి / Sign In", key="btn_signin"):
                ok, auth_user, msg = auth.authenticate_user(login_user, login_pin)
                if ok:
                    st.session_state.clear()
                    st.session_state["user"] = auth_user
                    st.success(msg)
                    st.rerun()
                else:
                    st.error(msg)

        with auth_tab_signup:
            st.markdown("#### కొత్త దుకాణం నమోదు (Create Account)")
            s_shop = st.text_input("దుకాణం పేరు (Shop Name)", placeholder="ఉదా: శ్రీ బాలాజీ కిరాణా స్టోర్స్", key="s_shop")
            s_owner = st.text_input("యజమాని పేరు (Owner Name)", placeholder="ఉదా: రమేష్", key="s_owner")
            s_user = st.text_input("యూజర్‌నేమ్ (Username / ID)", placeholder="ఉదా: rakesh_kirana", key="s_user")
            s_pin = st.text_input("4-అంకెల సెక్యూరిటీ పిన్ (PIN)", type="password", placeholder="కనీసం 4 అంకెలు", key="s_pin")

            s_lang = st.radio(
                "మీ ప్రధాన్య భాషను ఎంచుకోండి (Choose Default Language):",
                options=["te", "hi", "en"],
                format_func=lambda x: {"te": "తెలుగు (Telugu)", "hi": "हिंदी (Hindi)", "en": "English"}[x],
                key="s_lang"
            )

            if st.button("ఖాతను సృష్టించండి / Create Account", key="btn_signup"):
                if not s_user.strip() or not s_pin.strip():
                    st.error("Please enter Username and PIN.")
                else:
                    ok, msg = auth.register_user(s_user, s_shop, s_owner, s_pin, s_lang)
                    if ok:
                        st.success(msg)
                        # Ensure empty ledger file exists strictly for this new user
                        ledger.save_entries([], s_user.strip().lower())
                        _, new_user, _ = auth.authenticate_user(s_user, s_pin)
                        st.session_state.clear()
                        st.session_state["user"] = new_user
                        st.rerun()
                    else:
                        st.error(msg)

else:
    # --- USER IS LOGGED IN ---
    current_username = user["username"]
```

```
# Language resolution
current_lang = user.get("language", "te")
if current_lang not in I18N:
    current_lang = "te"
T = I18N[current_lang]

# --- SIDEBAR CONFIGURATION ---
st.sidebar.title(" " + user.get("shop_name", "Shop"))
st.sidebar.caption(f" ఖాతాదారు (Owner): **{user.get('owner_name', user['username'])}**")
st.sidebar.caption(f" Shop ID: `{current_username}`")

# Language Switcher
st.sidebar.markdown("---")
st.sidebar.subheader(f" {T['switch_lang']}")
lang_options = ["te", "hi", "en"]
lang_labels = {"te": "తెలుగు (Telugu)", "hi": "हिंदी (Hindi)", "en": "English"}

selected_lang = st.sidebar.selectbox(
    "Select Language",
    options=lang_options,
    index=lang_options.index(current_lang),
    format_func=lambda x: lang_labels[x],
    label_visibility="collapsed"
)

if selected_lang != current_lang:
    auth.update_language(current_username, selected_lang)
    user["language"] = selected_lang
    st.session_state["user"] = user
    st.rerun()

# Logout Button
if st.sidebar.button(f" {T['logout']}", type="secondary"):
    st.session_state.clear()
    st.rerun()

# Display ONLY this shop's registered customers in sidebar
st.sidebar.markdown("---")
st.sidebar.subheader(f"{T['shop_customers_title']}")
shop_customers = ledger.customers(current_username)
if shop_customers:
    for c in shop_customers:
        st.sidebar.markdown(f"• **{c}**")
else:
    st.sidebar.caption(f"_{T['no_customers']}_")

st.sidebar.markdown("---")
st.sidebar.subheader(f"{T['quick_demo']}")
st.sidebar.caption("Click any sample to test the NLP pipeline instantly:")

# Tailor sample names to this shop if customers exist, or provide fresh names
primary_cust = shop_customers[0] if shop_customers else ("రాము" if current_username == "chari" else "కస్టమర్")
secondary_cust = shop_customers[1] if len(shop_customers) > 1 else ("సీత" if current_username == "chari" else "రెండో_కస్టమర్")

sample_phrases = [
    ("Telugu: Rice (బియ్యం / BM)", f"{primary_cust} 500 రూపాయలు బియ్యం"),
    ("Telugu: Sugar (పంచదార)", f"{secondary_cust} 200 రూపాయలు పంచదార"),
    ("Telugu: Oil (నూనె)", f"{primary_cust} 180 రూపాయలు నూనె"),
    ("Telugu: Ask Balance (బాకీ)", f"{primary_cust} ఎంత బాకీ"),
```

```
    ("Telugu: Payment (చిల్లింపు)", f"{primary_cust} 200 చిల్లించడు"),
    ("Hindi: Flour (आटा)", f"{primary_cust} 300 रुपये आटा"),
    ("English: Correction", "make it 350"),
    ("English: Settle Full", f"{primary_cust} paid full"),
]

for label, phrase in sample_phrases:
    if st.sidebar.button(label, key="btn_" + label):
        st.session_state["proposal"] = pipeline.understand(phrase, current_username)
        st.session_state["proposal"]["language"] = "te-IN / hi-IN / en-IN"
        st.rerun()

st.sidebar.markdown("---")
st.sidebar.caption(f"Account: @{current_username} | Isolated Data Storage\nPowered by Gnani.ai Prisma v3.0")

# --- MAIN DASHBOARD HEADER ---
st.title(T["app_title"])
st.markdown(f"### *{T['app_subtitle']}*")
st.info(f" **{user.get('shop_name')}** (Owner: **{user.get('owner_name')}**) — Private Ledger Session")

# --- METRIC CARDS ---
metrics = ledger.financial_metrics(current_username)
balances = ledger.all_balances(current_username)
active_customers = len([b for b in balances.values() if b > 0])
settled_customers = len([b for b in balances.values() if b == 0])

col_m1, col_m2, col_m3, col_m4 = st.columns(4)
col_m1.metric(T["outstanding"], f"₹{metrics['current_outstanding']:,.0f}")
col_m2.metric(T["active_cust"], f"{active_customers}")
col_m3.metric(T["settled_cust"], f"{settled_customers}")
col_m4.metric(T["recovery_rate"], f"{metrics['recovery_rate']}%")

# --- SOUNDBOX NOTIFICATION BANNER ---
message = st.session_state.pop("message", None)
if message:
    if message["ok"]:
        st.success(f" {message['message']}")
        if message.get("soundbox_text"):
            components.html(tts.soundbox_html(message["soundbox_text"], current_lang), height=85)
    else:
        st.error(f" {message['message']}")

st.markdown("---")

# --- SECTION 1 & 2: VOICE INPUT & PROPOSAL ---
col_left, col_right = st.columns([1, 1])

with col_left:
    st.subheader(T["step1_title"])
    audio = st.audio_input(T["record_hint"])
    uploaded = st.file_uploader(T["or_upload"], type=["wav", "mp3", "m4a", "ogg", "flac", "aac"])
    clip = audio if audio is not None else uploaded
    if clip is not None:
        st.audio(clip)
        if st.button(T["btn_transcribe"], type="primary"):
            ext = os.path.splitext(getattr(clip, "name", "x.wav"))[1] or ".wav"
            path = "temp_clip" + ext
            with open(path, "wb") as f:
                f.write(clip.getvalue())
            try:
```

```
            with st.spinner("Listening via Gnani Prisma v3.0..."):
                st.session_state["proposal"] = pipeline.propose(path, current_username)
        except Exception as e:
            st.error(f"Error: {e}")
        if os.path.exists(path):
            os.remove(path)
        st.rerun()

with col_right:
    st.subheader(T["step2_title"])
    proposal = st.session_state.get("proposal")

    if proposal:
        st.info(f"**{T['heard']}** \"{proposal['transcript']}\"")
        st.markdown(f"**{T['intent']}** `{proposal['intent'].upper()}` | **{T['lang']}** `{proposal.get('language', 'Auto')}`")

        # Show issues if any
        if proposal["problems"]:
            st.warning(" Action required: " + ", ".join(proposal["problems"]))

            # Fallback if only customer name is missing
            if proposal["problems"] == ["customer name not found"]:
                typed = st.text_input("Type customer name to continue:")
                if st.button("Save with this name") and typed.strip():
                    proposal["entry"]["customer"] = typed.strip()
                    proposal["problems"] = []
                    st.session_state["message"] = pipeline.confirm(proposal, current_username, lang=current_lang)
                    st.session_state.pop("proposal")
                    st.rerun()
        else:
            entry = proposal["entry"]
            name_choice = entry.get("customer") or ""
            item_val = entry.get("item") or "—"
            amount_val = entry.get("amount") or "—"

            st.markdown(f"**{T['customer']}** `{name_choice}` | **{T['amount']}** `₹{amount_val}` | **{T['provision']}** `{item_val}`")

            if proposal["intent"] != "correct":
                similar = proposal.get("similar_to")
                if similar:
                    st.warning(f"Heard '{entry['customer']}'. Did you mean saved customer '{similar}'?")
                    options = [similar, entry["customer"]] if proposal["intent"] == "pay" else [entry["customer"], similar]
                    name_choice = st.radio("Select Customer:", options)
                else:
                    name_choice = st.text_input(f"{T['customer']} (edit if needed):", value=entry["customer"])

            c_btn1, c_btn2 = st.columns(2)
            if c_btn1.button(T["btn_confirm"], type="primary"):
                if proposal["intent"] != "correct" and name_choice.strip():
                    entry["customer"] = name_choice.strip()
                st.session_state["message"] = pipeline.confirm(proposal, current_username, lang=current_lang)
                st.session_state.pop("proposal")
                st.rerun()

            if c_btn2.button(T["btn_cancel"]):
                st.session_state.pop("proposal")
                st.rerun()
    else:
        st.caption("Record voice or click a quick demo button on the sidebar to preview understanding.")
```

```
    st.markdown("---")

    # --- SECTION 3: THE KHATA LEDGER ---
    st.subheader(T["step3_title"])

    summary = ledger.customers_summary(current_username)
    if summary:
        tab_balances, tab_history, tab_analytics = st.tabs([T["tab_balances"], T["tab_history"], T["tab_analytics"]])

        with tab_balances:
            # Search Filter
            search_query = st.text_input(T["search_label"], placeholder="Search by name or provision...").strip().lower()

            filtered_summary = summary
            if search_query:
                filtered_summary = [s for s in summary if search_query in s["Customer"].lower() or search_query in s["Provisions"].lower()]

            df_summary = pd.DataFrame(filtered_summary)
            if not df_summary.empty:
                # Localize status strings
                def map_status(val):
                    if "Settled" in val or "క్లియర్" in val:
                        return T["status_settled"]
                    elif "Owes" in val or "బాకీ" in val:
                        return T["status_owes"]
                    return T["status_advance"]

                df_summary["Status"] = df_summary["Status"].apply(map_status)

                df_display = df_summary.rename(columns={
                    "Customer": T["col_customer"],
                    "Balance": T["col_balance"],
                    "Provisions": T["col_provisions"],
                    "Status": T["col_status"]
                })
                st.dataframe(df_display, use_container_width=True)

                # 1-Click WhatsApp Reminder Section
                st.markdown("#### " + T["send_whatsapp"])
                debtors = [s for s in summary if s["Balance"] > 0]
                if debtors:
                    wa_col1, wa_col2 = st.columns([1, 1.5])
                    with wa_col1:
                        target_cust = st.selectbox("Select Customer to Send Reminder:", options=[d["Customer"] for d in debtors])
                    with wa_col2:
                        cust_data = next((d for d in debtors if d["Customer"] == target_cust), None)
                        if cust_data:
                            wa_url = tts.generate_whatsapp_link(
                                cust_data["Customer"],
                                cust_data["Balance"],
                                cust_data["Provisions"],
                                shop_name=user.get("shop_name", "కిరణా షాపు"),
                                lang=current_lang
                            )
                            st.markdown(f"""
                            <a href="{wa_url}" target="_blank" style="text-decoration:none;">
                                <button style="background-color:#25D366; color:white; border:none; padding:10px 20px; border-radius:8px; font-weight:bold; cursor:point
er; margin-top:24px; display:flex; align-items:center; gap:8px;">
                                    <span></span> Send WhatsApp Reminder to {target_cust} (₹{cust_data['Balance']})
                                </button>
                            </a>
```

```
                    """, unsafe_allow_html=True)
            else:
                st.info("No matching records found for your search query.")

        with tab_history:
            entries = ledger.load_entries(current_username)
            if entries:
                df_entries = pd.DataFrame(entries)
                st.dataframe(df_entries, use_container_width=True)

                # Export option
                csv_data = df_entries.to_csv(index=False).encode("utf-8")
                st.download_button(T["btn_export"], data=csv_data, file_name=f"khata_{current_username}.csv", mime="text/csv")

        with tab_analytics:
            st.markdown("###  " + T["tab_analytics"])
            st.caption("Visual business insights computed in real-time from your voice ledger entries.")

            c_an1, c_an2 = st.columns(2)

            with c_an1:
                st.markdown(f"#### {T['top_debtors_title']}")
                debtors_data = {s["Customer"]: s["Balance"] for s in summary if s["Balance"] > 0}
                if debtors_data:
                    df_chart = pd.DataFrame(list(debtors_data.items()), columns=["Customer", "Outstanding (₹)"]).set_index("Customer")
                    st.bar_chart(df_chart)
                else:
                    st.success(" All accounts are settled! Zero outstanding debt.")

            with c_an2:
                st.markdown(f"#### {T['top_provisions_title']}")
                prov_data = ledger.provisions_stats(current_username)
                if prov_data:
                    df_prov = pd.DataFrame(list(prov_data.items()), columns=["Provision", "Orders"]).set_index("Provision")
                    st.bar_chart(df_prov)
                else:
                    st.info("No provision entries recorded yet.")
    else:
        st.info(T["empty_ledger"])
```

5. 60-Second Demo Video Blueprint (Crushing the Competition)

Time	Visual Scene	Spoken Dialogue / Audio	Technical Focus Highlighte
0:00 - 0:10	Login screen. Choose Telugu, log	'While Chari Khata apps focus on theoretical inventory, Kh	Multi-tenancy & login/auth
0:10 - 0:25	Speak Telugu: '■■■■ 500 ■■■■